

_ =8

سالن %placeofmeeting خوارزمی دانشکده علوم ریاضی



جلسه دفاع از پروژه کارشناسی

زبان برنامه نویسی تابعی Haskell برای روش های عددی

سخنران: سید احسان حسینی

زمان:

مکان:

هیئت داوران

۱- دکتر رضا مختاری

۲-

۳-

چکیده:

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه صنعتی اصفهان
دانشکده علوم ریاضی

زبان برنامه نویسی تابعی Haskell برای روش های عددی

پروژه کارشناسی ریاضی کاربردی

سید احسان حسینی

استاد راهنما

دکتر رضا مختاری

۱۴۰۵



دانشگاه صنعتی اصفهان
دانشکده علوم ریاضی

پروژه کارشناسی ریاضی کاربردی آقای سید احسان حسینی

تحت عنوان

زبان برنامه نویسی تابعی Haskell برای روش های عددی

در تاریخ توسط کمیته تخصصی زیر مورد بررسی و تصویب نهایی قرار گرفت.

دکتر رضا مختاری

۱- استاد راهنمای پروژه

۲- استاد داور ۱

۳- استاد داور ۲

سرپرست تحصیلات تکمیلی دانشکده

به نام خداوند جان و خرد	کز این برتر اندیشه برگزید
خداوند نام و خداوند جای	خداوند روزی ده رهنمای
خداوند کیوان و گردان سپهر	فروزنده ماه و نایید و مهر
ز نام و نشان و گمان برتر است	مکارنده بر شده پیکر است
به بینندگان آفریننده را	نبینی مرغبان دو بیننده را
نیابد و نیراندیشه راه	که او برتر از نام و از جایگاه

- فردوسی

در آغاز، بر خود لازم می‌دانم که از زحمات بی‌دیغ و راهبانی‌های حکیمانه استاد راهبانی کراتقدر، جناب آقای دکتر رضامحتاری در راستای انجام این رساله تشکر و قدردانی نمایم. بی‌شک همراهی خاضعانه ایشان در طول دوره کارشناسی تا کارشناسی ارشد سنراوارسپاسی بیش از این است. از خانم دکتر محدثه رمضان‌ی که امر مشاوره این پایان نامه را به عهده داشتند، بسیار تشکر می‌کنم. همچنین از داوران گرامی، آقای دکتر مهدی روحانی و خانم دکتر مریم محمدی کمال پاس و اتنان را دارم.

از پدر و مادر مهربان و عزیزم، برادرم ایلین و خواهرم شهربانو که همواره مشوق و حامی من در سختی‌ها و دشواری‌های زندگی بوده‌اند، کمال تشکر را دارم. در پایان از تمامی استادان و دوستان، از جمله آقایان دکتر سید محمود منجکانی و دکتر فرید بهرامی، که در طول سالیان تحصیلی همواره در کنار من بوده‌اند و تجربه‌های ارزنده‌ای به اینجانب آموخته‌اند، بسیار سپاس گزارم.

علی فروزنده، هفتجانی ۱۴۰۳/۰۶/۰۴

کلیه حقوق مالکیت مادی و معنوی مربوط به این پروژه متعلق به دانشگاه صنعتی اصفهان و پدیدآورندگان است. این حقوق توسط دانشگاه صنعتی اصفهان و براساس خط مشی مالکیت فکری این دانشگاه، ارزش‌گذاری و سهم‌بندی خواهد شد. هر گونه بهره‌برداری از محتوا، نتایج یا اقدام برای تجاری‌سازی دستاوردهای این پروژه تنها با مجوز کتبی دانشگاه صنعتی اصفهان امکان‌پذیر است.

فروزندگان راه تاریک علم؛

آنان که برای بیداری انسان از خواب غفلت، رویاهای زندگی خود را فدا کردند.

فهرست مطالب

هفت	فهرست مطالب
نه	پیشگفتار
۱	۱ مقدمه
۱	۱.۱ مقدمه ای بر زبان های برنامه نویسی
۱	۲.۱ برخی پارادایم های رایج
۲	۳.۱ مقایسه دقیق تر پارادایم های تابعی و دستوری
۲	۴.۱ معرفی زبان Haskell
۲	۱.۴.۱ تاریخچه
۳	۲.۴.۱ ویژگی های مهم زبان
۶	نتیجه گیری و پیشنهادات
۷	کتاب نامه
۸	واژه نامه فارسی به انگلیسی
۱۱	واژه نامه انگلیسی به فارسی
۱۶	فهرست نمادها
۱۷	فهرست کلمات اختصاری

چکیده:

در این گزارش ابتدا به بررسی پارادایم های برنامه نویسی میپردازیم. سپس زبان Haskell را معرفی و مقدماتی بر این زبان ارایه میکنیم. ویژگی های زبان را بررسی میکنیم و نکاتی را هم درباره این زبان ذکر میکنیم. در نهایت به بررسی مثال هایی از کاربرد زبان Haskell در آنالیز عددی میپردازیم و مراجعی برای یادگیری عمیق تر زبان معرفی میکنیم.

پیشگفتار

در حل مسایل عددی زبان های برنامه نویسی نقش تائین کننده ای در نحوه پیاده سازی الگوریتم ها و روش های عددی دارند. پارادایم زبان، توانایی تشخیص و مدیریت خطاهای عددی یا باگ ها، میزان استفاده از حافظه و پردازشگر تحت تاثیر مستقیم زبان مورد استفاده هستند. اکثر زبان های مورد استفاده در محاسبات عددی از نوع imperative هستند. در این میان زبان های functional مثل Haskell کمتر استفاده میشوند. در این گزارش ابتدا مقدمه ای بر این زبان ارائه میکنیم و سپس ویژگی هایی از آن را بررسی میکنیم که میتوانند در حل مسایل کاربردی باشند و در کنار زبان هایی مانند MATLAB و C ابزار خوبی برای محققین باشد. زبان Haskell ویژگی های منحصر به فردی دارد که در آنرا تبدیل به ابزار مناسبی برای پژوهش میکند. در [؟] نویسندگان یک زبان (DSL) Domain Specific با Haskell میسازند تا یک زبان انتزاعی تر و ساده تر نسبت به CUDA C++ به دست آید. در نهایت با بررسی این زبان و مقایسه عملکرد آن با کد CUDA C++ نتیجه میگیرند که با کد های بسیار ساده تر میتوان به عملکردی مشابه به کد های CUDA C++ دست یافت. J. Karczmarszuk. چند مقاله در زمینه مشتق گیری خودکار یا Automatic Differentiation (AD) دارد که در آنها از زبان Haskell برای پیاده سازی روش های خود استفاده کرده. در [؟] [او از Lazy Evaluation و تکنیک های آن در تولید و استفاده از دنباله های طویل یا مشتقات عبارات عددی یک متغیره استفاده میکند.

فصل ۱

مقدمه

در این فصل زبان هارا از دیدگاه پارادایم بررسی میکنیم، اصطلاحات و تعاریف مورد نیاز بخش های بعدی را ارایه میکنیم.

۱.۱ مقدمه ای بر زبان های برنامه نویسی

واژه پارادایم تعریف دقیق و مشخصی ندارد اما راهی برای درک راحت تر و دسته بندی زبان های برنامه نویسی به دست میدهد. یک پارادایم برنامه نویسی یک شیوه یا نحوه نوشتن برنامه است. در هر زبانی شیوه هایی اسان تر و شیوه هایی سخت تر از دیگران است.

۲.۱ برخی پارادایم های رایج

پارادایم دستوری یا imperative رایج ترین است. در این نوع برنامه نویسی برنامه به صورت دنباله ای از دستورات صریح نوشته میشود. این دستورات در زمان اجرا تغییراتی در متغیر ها به وجود می آورند که به عنوان وضعیت یا state شناخته میشوند. در زبان های C, Python, MATLAB بیشتر از این شیوه استفاده میشود. این زبان ها اصطلاحاً Multi-paradigm هستند، به این معنی که میتوان در این زبان ها به شیوه های متعدد برنامه نوشت. در زبان های Python, MATLAB پارادایم های شی گرای و تابعی هم قابل استفاده هستند. در پارادایم شی گرا یا object-oriented برنامه با تعریف اشیا یا object ها که هرکدام وضعیت خود و توابعی که آن وضعیت را تغییر میدهند

نوشته میشوند. ایده اصلی روش شی گرا این است که داده ها و توابع با هم جفت باشند. روش تابعی یا functional بر این ایده مبتنی است که برنامه دیگر وضعیت کلی نداشته باشند و تمام برنامه در چهارچوب توابع نوشته شده باشد. حالت خاص تر پارادایم تابعی، تابعی محض یا Purely Functional است که در آن برنامه به هیچ عنوان وضعیت ندارد، حلقه های تکرار وجود ندارند و توابع میتوانند با هم ترکیب شوند، به صورت ورودی به توابع داده شوند و یا خروجی توابع دیگر باشند. زبان مورد بررسی این گزارش، یعنی زبان Haskell از دسته زبان های تابعی محض است.

۳.۱ مقایسه دقیق تر پارادایم های تابعی و دستوری

در برنامه نویسی دستوری جریان کنترل (Control Flow) صریح است. هر دستور نشان میدهد چگونه پردازش و محاسبات صورت میگیرد. هر دستور بر وضعیت عمومی برنامه تاثیر میگذارد. کامپیوتر های دیجیتال طراحی شده اند تا کد ماشین (Machine Code) اجرا کنند که خود عموماً به صورت دستوری نوشته میشود.

در برنامه نویسی تابعی برنامه ها توسط اعمال و ترکیب توابع ساخته میشوند. در این روش توابع مشابه توابع ریاضی درخت هایی از عبارات هستند که مجموعه ای از ورودی های قابل قبول را به مجموعه ای از خروجی ها میبرند. همانطور که اشاره شد حالت خاص روش تابعی، تابعی محض است. در این روش توابع برنامه تفاوت بزرگی با روش دستوری درند و آن در این است که همه توابع برنامه به ازای یک ورودی یک خروجی منحصر به فرد دارند. به عبارت دیگر خوش تعریف هستند. در زبان های دستوری توابع میتوانند خوش تعریف نباشند. برای مثال ورودی x به تابع f خروجی $y = f(x)$ میدهد اما با یک بار دیگر ورودی دادن x به تابع f میتوان خروجی $z = f(x)$ را دریافت کرد. در زبان هایی مانند C این رفتار ها بعضاً باعث بروز باگ میشوند. باگ هایی که در زبان های تابعی محض به هیچ عنوان رخ نخواهند داد.

۴.۱ معرفی زبان Haskell

به منظور درک بهتر زبان در این بخش گریزی به تاریخچه زبان Haskell میرنیم و برخی کاربردهای آن را در صنعت فهرست میکنیم.

۱.۴.۱ تاریخچه

پس از انتشار زبان Miranda (یک زبان تابعی محض) در سال ۱۹۸۵ توسط شرکت Research Software Ltd. توجه به زبان های تنبل بسیار زیاد شد. تا سال ۱۹۸۵ بیش از ۱۰ زبان تابعی ساخته شده و مورد استفاده بودند

که هرکدام پیاده سازی ها و برداشت های متفاوتی در نحوه بهینه طراحی زبان ها داشتند. این موضوع در کنار تجاری بودن زبان Miranda باعث شد تا در کنفرانس Programming Languages and Computer Architecture (FPCA '87) پیشنهاد تشکیل یک کمیته برای استاندارد سازی نحوه طراحی و ساخت زبان های مشابه طرح شد. وظیفه این کمیته طراحی زبانی مشترک بود که پایه ای برای تحقیقات آینده در حوزه طراحی زبان های تابعی باشد. Haskell توسط یک گروه توسعه داده شد که تلاش میکرد از روش های بیشتر مطالعه شده استفاده کند. مفهوم Type Class ها برای اولین بار توسط Phillip Walder, Stephen Blott برای حل مشکلات تساوی ها و حساب انواع مختلف متغیر ها که زبان های موجود آن زمان را با مشکلاتی رو به رو کرده بود. زبان های برنامه نویسی موجود در آن زمان هرکدام راه متفاوتی را برای بررسی تساوی و حساب رفته بودند. مثلاً زبان Standard ML عملگر جمع در کامپایلر برای متغیر های ممیز شناور و صحیح Float و Integer تعریف شده بود اما کاربر هیچ راهی برای تعریف عملگر جمع برای متغیر های دیگر نداشت. این موضوع برای بررسی تساوی میان دو متغیر هم صادق بود. Type Class ها راه حلی برای این مشکل بودند. به مرور زمان استاندارد زبان Haskell تکامل پیدا کرد و همچنان در حال توسعه است. استاندارد Haskell 2010 در حال حاضر تعریف رسمی این زبان است.

۲.۴.۱ ویژگی های مهم زبان

صفحه اصلی زبان ویژگی های زیر را علاوه بر Lazy Evaluation و Purely Functional را فهرست میکند. در اینجا به بررسی آنها میپردازیم.

این زبان ایستا گونه یا Statically Typed است به این معنی که نوع متغیر ها از قبل مشخص و در زمان اجرای برنامه ثابت است. زبان Haskell با زبان C که ایستا گونه است در این موضوع متفاوت است که نوع تمام عبارات هم در زمان کامپایل مشخص میشود. اگر کامپایلر تشخیص دهد در جایی از برنامه انواع با هم همخوانی ندارند برنامه را کامپایل نمیکند. زبان Haskell دارای ویژگی Type Inference یا استنباط نوع است به این معنی که میتواند درباره نوع عبارات به صورت خودکار بدون دخالت کاربر استدلال کند و تشخیص دهد عبارت چه نوعی دارد.

```
main :: IO ()
main = do line :: String <- getLine
        print (parseDigit line)
    where parseDigit :: String -> Maybe Int
          parseDigit ((c :: Char) : _) =
            if isDigit c
            then Just (ord c - ord '0')
            else Nothing
```

عمل این برش کوتاه کد در اینجا مورد بحث نیست، اما در هرجه که نماد :: آمده نوع متغیر یا عبارت مشخص میشود. مثلاً در `parseDigit :: String -> Maybe Int` نام `parseDigit` نوع تابعی که رشته دریافت و نوع `Maybe Int`

خروجی میدهد را میگیرد. ویژگی استنباط نوع باعث میشود در بسیاری از موارد دیگر نیازی به مشخص کردن نوع نباشد. کد زیر معادل کد بالاست.

```
main = do line <- getLine
        print (parseDigit line)
  where parseDigit (c : _) =
        if isDigit c
        then Just (ord c - ord '0')
        else Nothing
```

این زبان دارای کتابخانه سبکی برای برنامه نویسی موازی است. بررسی برنامه نویسی موازی در چهارچوب این گزارش نیست، به همین علت به آن نمیپردازیم و صرفاً به ذکر این مطلب بسنده میکنیم که ویژگی های این زبان باعث میشوند برنامه نویسی موازی نقطه قوت آن باشد.

Haskell از استراتژی ارزیابی کندرو با Lazy Evaluation استفاده میکند. در این استراتژی، ارزیابی عبارات تا جای ممکن به تاخیر می افتند. این استراتژی موجب کاهش استفاده از حافظه در هر لحظه خواهد شد و با تکنیک هایی مثل Memoization میتوان به برنامه سرعت بخشید. در مقابل این استراتژی ترتیب برآورد شدن عبارات را بهم میریزد در نتیجه مدیریت استثناها و ورودی/خروجی ها با مشکلاتی رو به رو میشوند. علاوه بر آن میتواند منجر به نشت حافظه شود.

زبان Haskell دارای ۶۹۵۴ کتابخانه، موتور جستجوی **Hackage** برای کتابخانه ها و زبان، و ابزارهای Cabal و Stack برای ساخت و کامپایل برنامه هاست.

نتیجه‌گیری و پیشنهادات

معادله SFD یک مسئله چالشی با کاربردهایی در مدل‌سازی انتشار غیرعادی، بررسی پدیده‌های زیرانتشار و توصیف دینامیک‌های آشوبی است. چالش اصلی در حل عددی این معادله، غیرموضعی بودن عملگر کسری است که به تولید یک دستگاه پر در گسسته‌سازی مکانی منجر می‌شود و حجم محاسبات در هر گام زمانی را افزایش می‌دهد. از طرفی با توجه به تعریف مشتق کسری، دقت محاسبات مربوط به هر درایه این دستگاه خود یک مشکل جدی است.

کتاب نامه

(

واژه‌نامه فارسی به انگلیسی

ا

Underlying اساسی

Temporal propagator انتشاردهنده زمانی

پ

Marching پیشرو

ت

Simultaneous توأم

Distributed توزیع یافته

چ

Piecewise polynomials چندجمله‌ای‌های قطعه‌ای

Cycle چرخه

د

Coarse درشت

Global interpolation درون‌یاب سراسری

Local interpolation درون‌یاب موضعی

Bilinear دوخطی

ر

Cluster computer رایانه خوشه‌ای

ز

Essential supremum زیرینه اساسی

Subdiffusion زیرانتشار

س

Hierarchy سلسله مراتب

ش

Mesh, Grid شبکه

ظ

Fine ظریف

ف

Parareal فراواقعی

Test Space فضای آزمایش

Trial Space فضای آزمون

ق

Simultaneous diagonalization قطری‌شونده توأم

ک

Total-reduction کاهش کلی

Alternating-reduction کاهش تناوبی

م

Restriction محدودیت

Support محمل

Scalabe مقیاس‌پذیر

و

Coercive وادارنده

ه

Concurrency همروندسازی

Smooth هموار

واژه‌نامه انگلیسی به فارسی

A

Affine	آفین
Analytic function	تابع تحلیلی

B

Boundary component	مولفه مرزی
--------------------------	------------

C

Calculation	محاسبات
Center	مرکز
Change	تبدیل
Closed orbit	مدار بسته
Coefficient	ضریب
Compactness	فشردگی
Continuous	پیوسته
Contradiction	تناقض
Coordinates	مختصات
Corollary	نتیجه
Counted with multiplicities	با احتساب تکرار
Criteria	معیار

Critical period دوره بحرانی
Curve منحنی

D

Dehomogenized غیرهمگن
Denominator مخرج
Divid تقسیم
Discriminant مبین

E

Energy level سطح انرژی
Explicit صریح

F

Factor عامل
First integral انتگرال اول
Foliated افراز (یا ورقه‌بندی) شده
Function تابع

H

Hamiltonian system دستگاه همیلتونی
Half-plane نیم صفحه

I

Implicitly	به صورت ضمنی
Integrating factor	عامل انتگرال ساز
Interval	بازه
Invariant	ناوردا
Involution	برگردان
Isochronous	هم زمان-هم دوره

L

Lemma	لم
-------------	----

M

Multiplicities	چندگانگی ها
Maple	مپل
Mapping	نگاشت
Maximum	بیشینه
Minimum	کمینه
Monotone	یکنوا
Monotonicity	یکنوایی
Multiply	ضرب

N

Negative	منفی
Neighborhood	همسایگی
Notation	نماد

P

Parameter	پارامتر
Period annulus	طوق تناوبی
Period annuli	طوق‌های تناوبی
Period function	تابع دوره تناوب
Polynomial	چندجمله‌ای
Polycycle	چندسیکل
Potential system	دستگاه پتانسیل
Procedure	روش
Proof	اثبات
Proposition	گزاره
Punctured	محذوف

R

Regular	منظم
Rescaling	تغییر مقیاس
Resultant	برآیند
Reversible	برگشت‌پذیر
Root	ریشه

S

Sign	علامت
Statement	گزاره
Surrounding	حول

T

Tangent مماس

Theorem قضیه

Transformation تبدیل

True درست

U

Unique یکتا

V

Variation method روش تغییرات

W

Well defined خوش تعریف

فهرست نمادها

مفهوم	نماد
اعداد حقیقی	$\mathbb{R}$
اعداد حقیقی مثبت	$\mathbb{R}^+$
اعداد طبیعی	$\mathbb{N}$
اعداد فرد	$\mathbb{O}$

فهرست کلمات اختصاری

کلمه کامل	معنی	اختصار
Finite elements method	روش عناصر متناهی (المان محدود)	FEM
Fractional partial differential equations	معادلات با مشتقات جزئی کسری	FPDEs
Space-fractional diffusion	انتشار کسری-مکانی	SFD
Multigrid reduction	کاهش چندشبکه‌ای	MGR
Multigrid reduction-in-time	کاهش در زمان چندشبکه‌ای	MGRIT

Multigrid reduction-in-time algorithms for solving fractional partial differential equations

Ali Forouzandeh Hafshejani
aforouzandeh@math.iut.ac.ir

August 31, 2024

Master of Science Thesis (in Farsi)

Departement of Mathematical Sciences

Isfahan University of Technology, Isfahan 84156-8311, Iran

Supervisor: Dr. Reza Mokhtari, mokhtari@iut.ac.ir

Advisor: Dr. Mohadese Ramezani, mohadeseh.ramezani@alumni.iut.ac.ir

2020 MSC: 65M55, 65M60, 65Y05.

Keywords: Fractional calculus, Space-fractional diffusion, FEM, Parallel-in-time, Multigrid reduction-in-time.

Abstract:

This M.Sc. thesis is based on the following paper

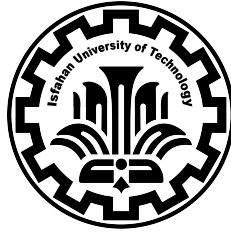
- Yue, X., Shu, S., Xu, X., Bu, W., & Pan, K. (2019). Parallel-in-time multigrid for space–time finite element approximations of two-dimensional space-fractional diffusion equations. *Comput. Math. Appl.*, 78. 3471–3484.

Nowadays, temporal or spatial fractional partial differential equations (FPDEs) have found applications in real-world problems in science and engineering. This popularity stems from the fractional derivative’s nonlocal property compared to the local nature of the integer-order derivative. One of the most important FPDEs is the space-fractional diffusion equation (SFD) which has applications in modeling anomalous diffusion, investigating subdiffusive phenomena, and describing chaotic dynamics. SFD over two-dimensional spaces is widely recognized as a key diffusion equation. It is derived from generalizing the spatial derivatives from integer order to fractional order within the partial differential equation. Since most SFD equations can not be solved analytically, various numerical methods, such as the finite difference method, local discontinuous Galerkin approach, and finite element method (FEM), have been proposed to achieve both high accuracy and efficiency. It is important to note that fractional derivatives use global information, while classical derivatives rely on local information. As a result, regardless of the discretization method used, significant computational effort is required due to the nonlocality introduced by fractional differential operators. Many researchers have worked on developing fast algorithms to address this challenge. In addition to these rapid solutions, parallel computing approaches, such as multigrid reduction in time (MGRIT), should also be considered potential techniques.

In this thesis, we begin by discussing fundamental concepts in functional analysis, including vector spaces, function spaces, and Sobolev spaces, as well as the principles of fractional calculus. We explain that fractional derivatives and integrals are foundational to fractional calculus, with the Ritz fractional derivative being particularly favored for applications in spatial domains. Next, we examine various fractional spaces, such as fractional Sobolev spaces, and investigate their properties. Additionally, we address spaces associated with FPDEs.

After that, we investigate the SFD problem with Dirichlet boundary conditions. We start by explaining the FEM and its properties. We construct the weak form of the SFD equation and apply space-time discretization, utilizing uniform spatial discretization and non-uniform temporal discretization. This process results in a large, sparse system of equations. To solve the SFD equation numerically, we represent the method as a time-marching loop, where a spatial linear system is solved at each time step. This time-marching loop acts as a one-step temporal method, similar to solving a lower bidiagonal block system over time. We also discuss various schemes for temporal parallelization and provide a brief historical context. One notable technique is MGRIT, which utilizes a multigrid reduction approach. The MGRIT offers two significant advantages: it minimizes interference with existing codes and allows optimal scalability. Subsequently, we employ a two-stage version of the MGRIT method to solve the lower bidiagonal system of the block unit and analyzing its convergence performance.

Finally, we implement a numerical example in MATLAB and XBraid and examine some tests. The results indicate that the method demonstrates adequate consistency and convergence for numerical solutions of such SFD equations, and it can be extended to solve some complicated FPDEs.



Isfahan University of Technology
Department of Mathematical Sciences

The Haskell Functional Programming Language for Numerical Methods

A Thesis
Submitted in Partial Fulfillment of the Requirements
for the Bachelor of Science (B. Sc.)

By
S. Ehsan Hoseini

Evaluated and approved by the thesis committee, on

1- Dr. Reza Mokhtari, Professor (Supervisor)

2- , (Examiner)

3- , (Examiner)

, (Department graduate coordinator)



Isfahan University of Technology
Department of Mathematical Sciences

Thesis Submitted for the Award of Bachelor of Science (B. Sc.) in Mathematics

The Haskell Functional Programming Language for Numerical Methods

S. Ehsan Hoseini

Supervisor: Dr. Reza Mokhtari